2026/02/24 - 2026/02/26
QI4U in PNU

Optimization for evacuation route of IT building

Team 04
모상훈, 강현택, 김태훈(Leader), 박지성
Mo Sanghun, Kang Hyentaek, Kim Taehun(Leader), Park Jisung
Mentor: Moriaki Shimazaki

■ Group Theme

Optimization for evacuation route of IT building

- **Target Environment:** IT Building
- **Core Objective:** Deriving real-time dynamic evacuation routes to secure the "Golden Time" during disasters.
- **Core Technology:** Introducing Quantum Annealing to overcome the computational limitations of multi-agent path optimization.

■ Background

Environmental Characteristics & Risks of IT Buildings

Complex Spatial Configuration & Fire Hazards:

- High concentration of large lecture halls (holding hundreds of students) and 24/7 research labs operating high-power servers.
- High risk of electrical fires due to heavy power consumption, leading to the rapid generation and spread of toxic smoke.

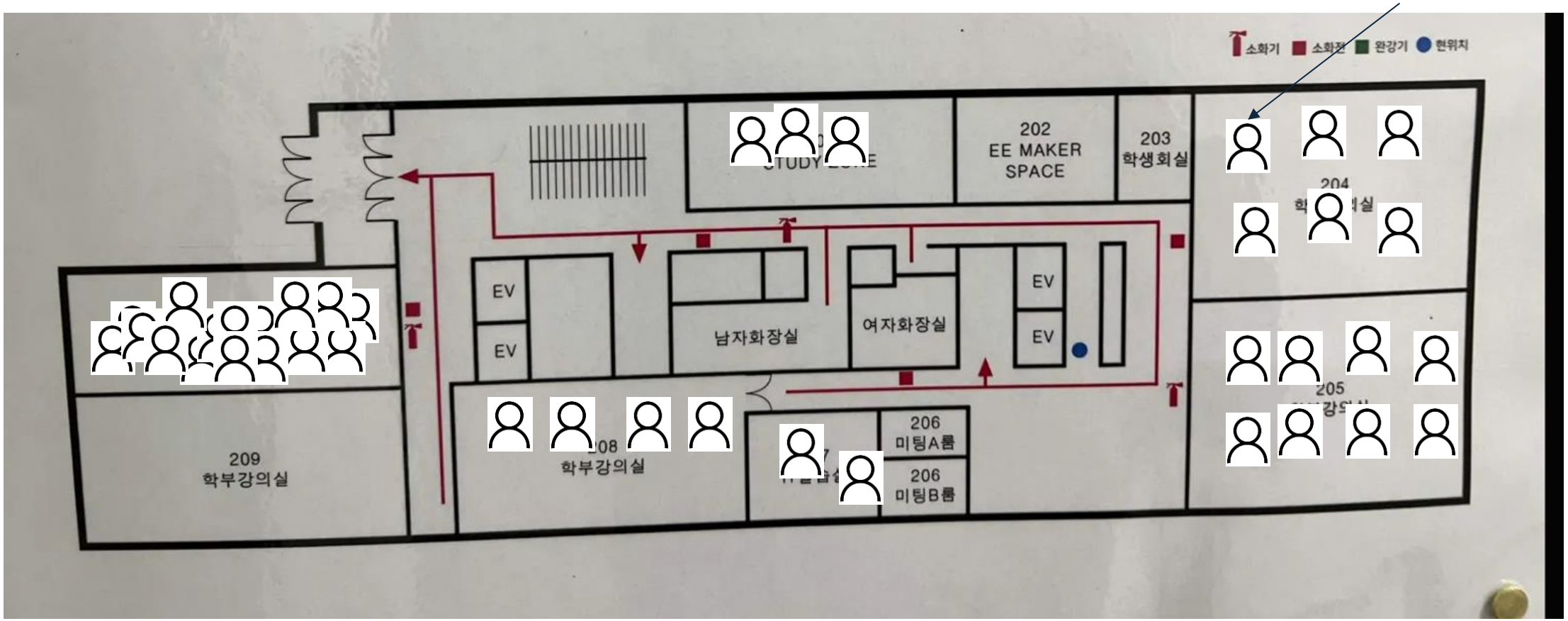
Extreme Time-Dependent Population Density (Dynamic Density):

- Severe congestion occurs in hallways and stairwells during class transitions as hundreds of students move simultaneously.
- Unpredictable population distribution during nights and weekends, with only a few researchers present in specific labs.

■ Background

Environmental Characteristics & Risks of IT Buildings

where should they go?



■ Problem Definition

Limitations of Current Evacuation Systems and Classical Algorithms

Fatal Flaws of Static Guidance:

- Current exit signs only indicate the "shortest physical distance," potentially leading evacuees directly toward the fire or smoke.

Bottlenecks and Secondary Casualties:

- Crowd concentration at specific main stairways or exits causes severe evacuation delays and risks secondary injuries (e.g., crushing in narrow spaces).

Computational Limits of Classical Computing (NP-Hard Problem):

- Calculating optimal detour routes for hundreds of individuals simultaneously while accounting for real-time fire spread is a problem with exponentially increasing variables.
- Finding a solution within the critical "Golden Time" is practically impossible using traditional computers.

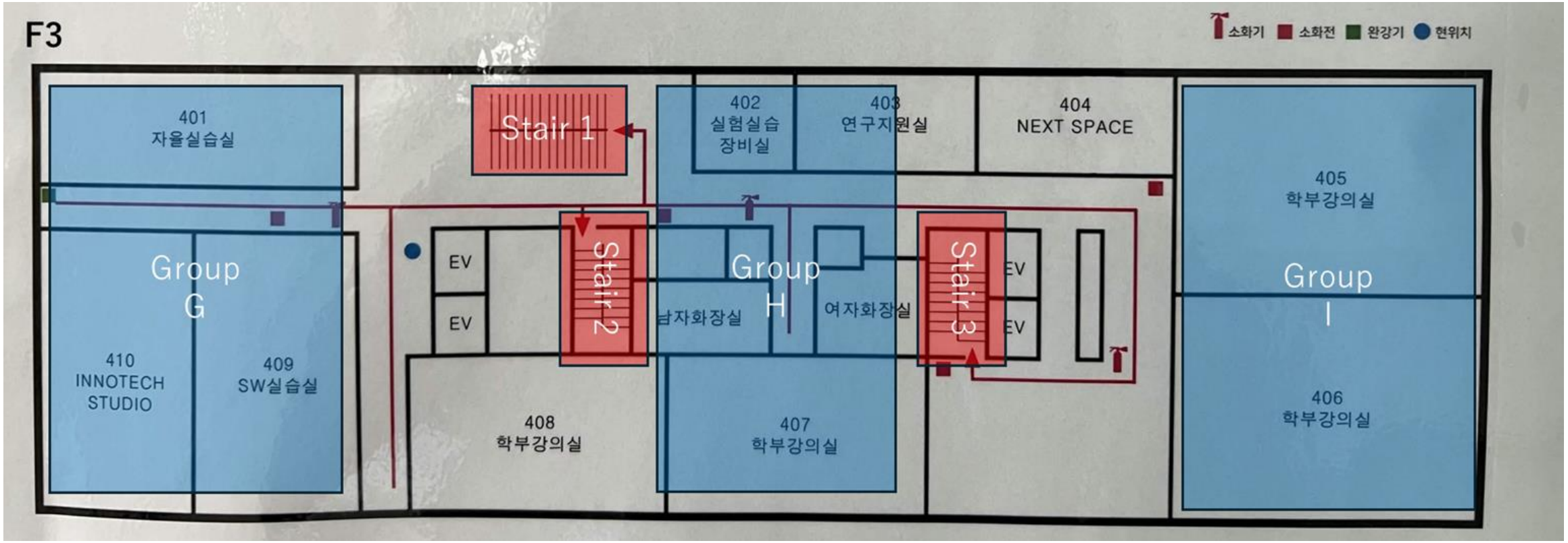
■ Motivation

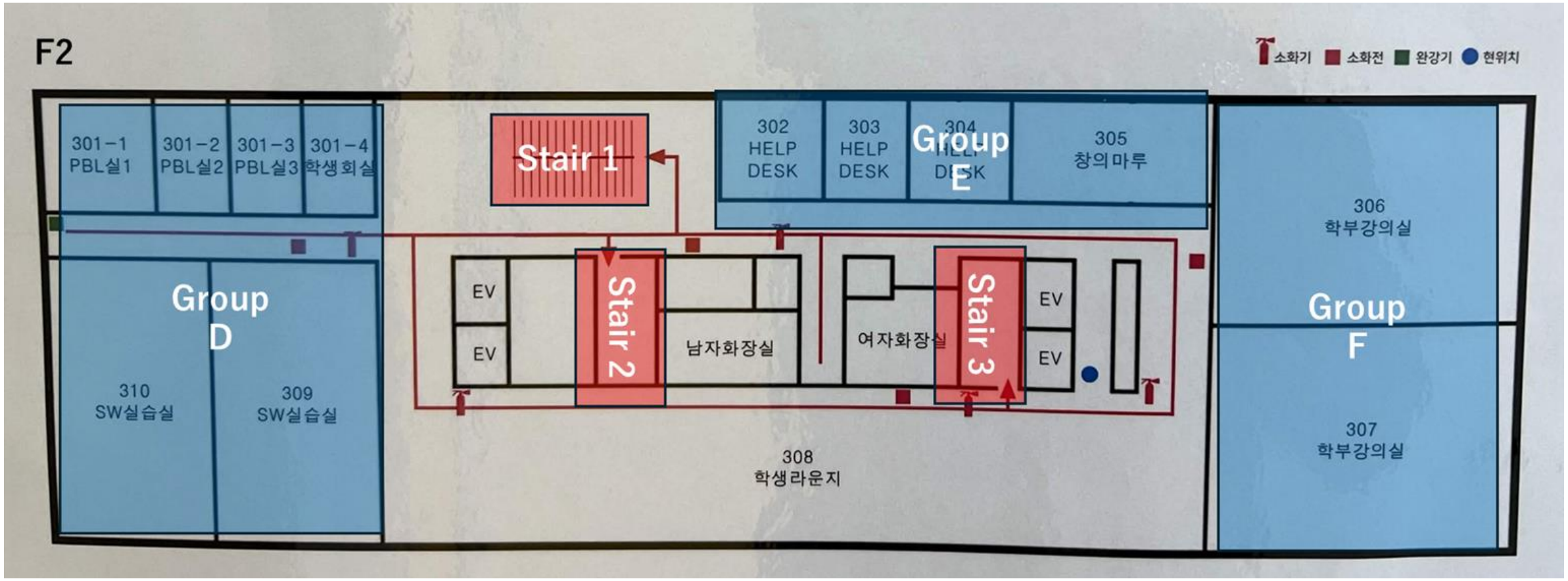
The Need for Real-Time Multi-Agent Optimization:

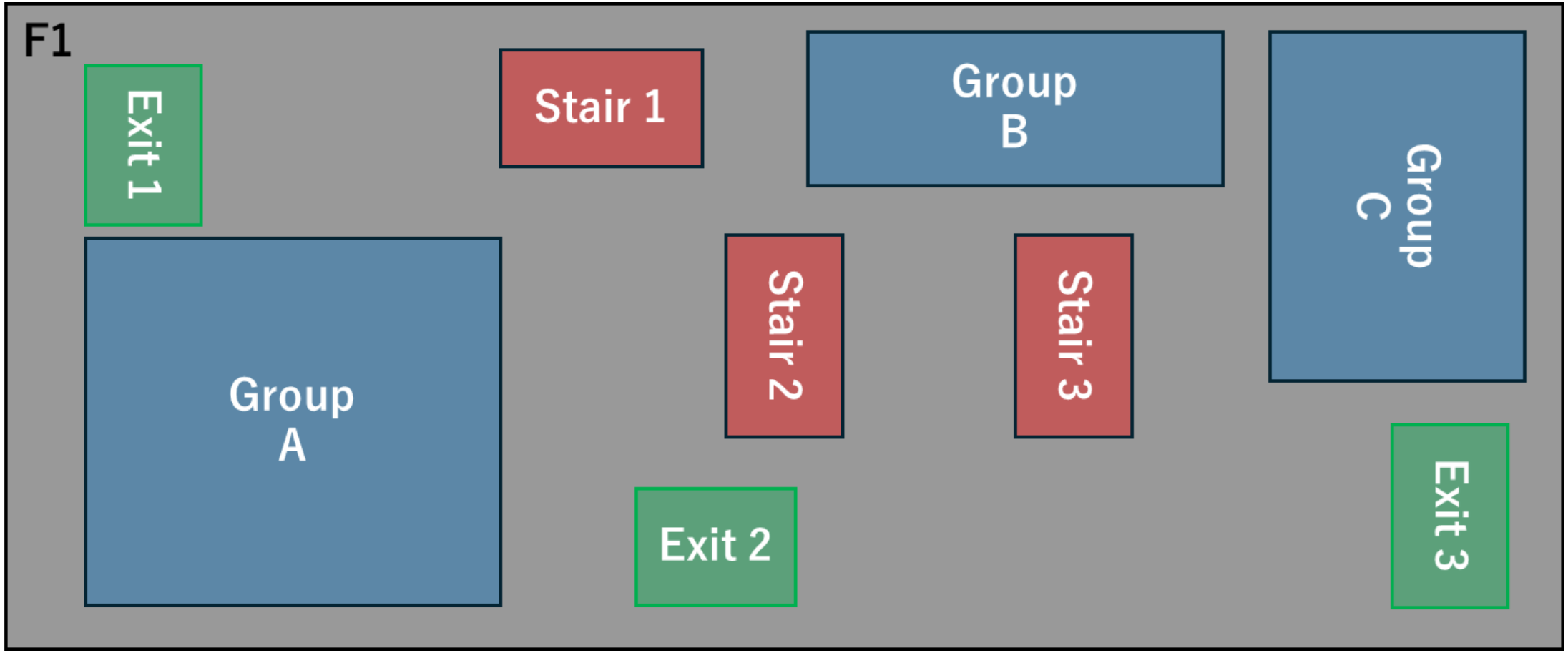
- The system must simultaneously coordinate the paths of all evacuees to prevent overlapping routes and bottlenecks.

A Paradigm Shift via Quantum Computing:

- Utilizing the characteristics of quantum annealing to simultaneously explore a vast solution space and find the Global Optimum instantly.
- Aiming to provide immediate survival routes without computational delay, leveraging crowd density data gathered from existing campus infrastructure (e.g., Wi-Fi AP connections).


F3
소화기
소화전
완강기
현위치
401
자율실습실
Stair 1
402
실험실습
장비실
403
연구지원실
404
NEXT SPACE
405
학부강의실
Group
G
EV
EV
Stair 2
남자화장실
Group
H
여자화장실
Stair 3
EV
EV
Group
I
410
INNOTECH
STUDIO
409
SW실습실
408
학부강의실
407
학부강의실
406
학부강의실


F2
소화기
소화전
완강기
현위치
301-1
PBL실1
301-2
PBL실2
301-3
PBL실3
301-4
학생회실
Stair 1
302
HELP
DESK
303
HELP
DESK
304
HELP
DESK
Group
E
305
창의마루
306
학부강의실
Group
D
310
SW실습실
309
SW실습실
EV
EV
Stair 2
남자화장실
여자화장실
Stair 3
EV
EV
Group
F
307
학부강의실
308
학생라운지


F1
Exit 1
Stair 1
Group
B
Group
C
Group
A
Stair 2
Stair 3
Exit 2
Exit 3

Framework

Quadratic Unconstrained Binary Optimization(QUBO):
Minimize H without constraints(instead, we use the penalty term)

```
QUBO

array([[ 0.68351357,  0.03326681, -1.10727539,  1.90360814,  0.08037236,
         0.75524003,  0.8079373 , -1.3906577 , -1.76223468, -1.46152524],
       [ 0.        , -1.01198512, -1.39450275,  0.3525127 ,  2.11950222,
         0.69840443, -0.50155932,  0.24645373, -0.16904969, -0.86374067],
       [ 0.        ,  0.        ,  0.79375591, -1.80350319,  1.87839607,
         0.65309533,  1.12593156, -0.43469862, -0.30833721,  0.29215849],
       [ 0.        ,  0.        ,  0.        ,  0.09800884,  1.43420272,
         0.79901768,  0.08934122,  0.04922748,  0.08903366, -0.23009227],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  1.07174299,
         0.43846064, -1.70894234, -0.60539435, -0.47981908,  0.48696028],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        -0.06399653,  0.74938904, -1.19070107,  0.05586424, -2.10361983],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        , -0.55955861, -0.25578475,  0.29070389, -0.12238039],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.77071296, -0.92672073, -0.73798605],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.        ,  0.76792623,  1.13490921],
       [ 0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
         0.        ,  0.        ,  0.        ,  0.        , -0.70752155]])
```

```
from neal import SimulatedAnnealingSampler
sa_sampler = SimulatedAnnealingSampler()

sampleset = sa_sampler.sample_qubo(QUBO, num_reads=10)

print(sampleset.record)

[([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)
 ([1, 1, 1, 0, 0, 1, 0, 1, 1, 1], -8.74925188, 1)]
```

Figure 1. QUBO

■ QUBO Formulation for Evacuation Route Optimization- Method 1

1. Variables and Constants

- **Decision Variable**
  - $x_{i,\mu}$ is a binary variable representing whether individual i select route $\boldsymbol{\mu}$
    - 1 if individual i chooses route μ, otherwise 0
- **Constant** $C_{\mu e}$
  - $C_{\mu e}$ indicates if route μ includes edge e
    - 1 if route μ passes through edge e, otherwise 0
- **Constant** T_μ
  - T_μ is the total travel time for route μ
  - Sum of travel times t_e for all edges e in the route

$$T_\mu = \sum_e C_{\mu e} t_e$$

■ QUBO Formulation for Evacuation Route Optimization- Method 1

2. The Objective Function(Hamiltonian)

- **The total Hamiltonian H is the sum of the constraint terms and the cost terms**
  - $H = H_{select} + H_{time} + H_{congestion}$
- **(1) Selection Constraint(One route per person)**
  - Each individual i must select exactly one route from their set of candidate route M_i

$$H_{select} = A \sum_{i} \left(\sum_{\mu \in M_i} x_{i,\mu} - 1 \right)^2$$

- **(2) Cost Term: Minimizing Travel time of all individuals to minimize the average time**
  - $H_{time} = B \sum_{i} \sum_{\mu \in M_i} T_\mu \, x_{i,\mu}$
- **(3) Congestion Avoidance (capacity constraint)**
  - To prevent overcrowding, we penalize the overlapping of routes on the same edge e

$$H_{congestion} = C \sum_{e} \left(\sum_{i} \sum_{\mu \in M_i} C_{\mu e} \, x_{i,u} \right)^2$$

■ QUBO Formulation for Evacuation Route Optimization- Method 1

3. Minimax approach (minimizing the last person's arrival)

- $H_{MINIMAX} = B \cdot T_{goal} + D \sum_i \left(\sum_{\mu \in M_i} T_\mu x_{i,\mu} - T_{goal} + s_i \right)$
- T_{goal} represents the bottleneck time(the global deadline),
- s_i is the slack variables to satisfy the inequality individual time $\leq T_{goal}$

■ QUBO Formulation for Evacuation Route Optimization- Method 2

1. Model the Building as a graph

- Nodes are classrooms, corridor junction, stair entrance
- Edges are hallway. Each edge has:
  - Length L_e
  - travel time τ_e
  - Capacity C_e
- Group people into units for calculation
- Goals are
  - Minimize time of the last evacuee
  - Minimize average evacuation time

■ QUBO Formulation for Evacuation Route Optimization- Method 2

2. Constraints

- Let the variable $x_{k,e,t} = \begin{cases} 1 & \text{group } k \text{ uses edge } e \text{ starting at time } t \\ 0 & \text{otherwise} \end{cases}$
- The total Hamiltonian H is the sum of the constraint terms and the cost terms
  - $H = H_{flow} + H_{cap} + H_{obj}$
- (1) Flow continuity (must form a valid path)
  - Each group must:
    - start at its origin
    - follow continuous edges
    - end at an exit
    - never teleport or branch

■ QUBO Formulation for Evacuation Route Optimization- Method 2

2. Constraints (cont'd)

- (1) Flow continuity (must form a valid path) (cont'd)
  - $$\sum_{\text{in}} x - \sum_{\text{out}} x = \begin{cases} -1, & (v,t) = \text{source} \\ +1, & (v,t) = \text{sink} \\ 0, & \text{otherwise} \end{cases}$$
  - Except for the start node and destination node, the number of groups entering edge e in the previous time slot and the number of groups exiting edge e in the current time slot must be the same
    - for start node, $b_{k,v,t}$ is +1, for destination node, $b_{k,v,t}$ is -1
    - Therefore, Flow continuity constraints is:

$$H_{flow} = A \sum_{k,v,t} \left(\sum_{e \in In(v,t)} x_{k,e,t-\tau_e} - \sum_{e \in Out(v,t)} x_{k,e,t} - b_{k,v,t} \right)^2$$

For example, (not enter, leave) means $(1-0)^2 = 1$, (enter, leave) means $(1-1)^2 = 0$

■ QUBO Formulation for Evacuation Route Optimization- Method 2

2. Constraints (cont'd)

- (2) Corridor capacity (avoid congestion)
  - For every corridor edge e and each time layer t:
    - $\sum_k x_{k,e,t} \leq C_e$
    - Therefore, $H_{cap} = A_{cap}\left(\max\left(0,\ \sum_k x_{k,e,t} - C_e\right)\right)^2$
    - However, QUBO cannot control max operator, we introduce slack variable
      - $\sum_k x_{k,e,t} + s_{e,t} = C_e$
      - $H_{cap} = A_{cap}\left(\sum_k x_{k,e,t} + s_{e,t} - C_e\right)^2$

- (3) Minimize average evacuation time

$$H_{obj} = B\sum_k T_k$$

■ Comparison between Method 1 and Method 2

Table 1. Comparison between method 1 and method 2

	Method 1	Method 2
Modeling Approach	Predefine candidate routes for each individual and select one optimal route	Model the building as a graph and dynamically construct paths via flow variables
Decision Variable	$x_{i,\mu}$ 1 if individual i selects route u	$x_{k,e,t}$ 1 if group k uses edge e at time t
Main Constraints	One route per person Travel time minimization Congestion penalty on shared edges	Flow continuity (valid path formation) corridor capacity per time step Travel time minimization
Path Structure	Routes are precomputed	Paths are generated through flow continuity constraints
Strength	Simpler formulation and computationally lighter	More accurate modeling of congestion and time-dependent flow
Weakness	May miss better paths not included in candidate set	Larger computational burden

■ Results

```python
# 1. Configuration values
groups = {
    "G": 20, "H": 30, "I": 20,  # 3rd floor
    "D": 40, "E": 60, "F": 10,  # 2nd floor
    "A": 30, "B": 200, "C": 10  # 1st floor
}

# Base time for each route [Option 1, Option 2, Option 3]
base_times = {
    "3F": [20, 25, 30],  # Base times for 3rd floor groups
    "2F": [15, 20, 25],  # Base times for 2nd floor groups
    "1F": [5, 10, 15]    # Base times for 1st floor groups
}

alpha = 0.2  # Delay coefficient based on population (adds 0.2 seconds per person)
```

```python
# 2. Build route information
route_options = {}
for g, pop in groups.items():
    if g in "GHI":
        floor = "3F"
    elif g in "DEF":
        floor = "2F"
    else:
        floor = "1F"

    for idx in range(3):
        # Dynamic time calculation: base time + (population × coefficient)
        calc_time = base_times[floor][idx] + (pop * alpha)

        # Define passing edges
        if floor == "3F":
            edges = [f"Stair_{idx+1}_3-2", f"Stair_{idx+1}_2-1", f"Exit_{idx+1}"]
        elif floor == "2F":
            edges = [f"Stair_{idx+1}_2-1", f"Exit_{idx+1}"]
```

```
Group  | Selected Route                  | Total Time | Pop
-----------------------------------------------------------------
G      | ['Stair_2_3-2', 'Stair_2_2-1', 'Exit_2'] | 29.0       | 20
H      | ['Stair_3_3-2', 'Stair_3_2-1', 'Exit_3'] | 36.0       | 30
I      | ['Stair_1_3-2', 'Stair_1_2-1', 'Exit_1'] | 24.0       | 20
D      | ['Stair_2_2-1', 'Exit_2']       | 28.0       | 40
E      | ['Stair_3_2-1', 'Exit_3']       | 37.0       | 60
F      | ['Stair_3_2-1', 'Exit_3']       | 27.0       | 10
A      | ['Exit_2']                      | 16.0       | 30
B      | ['Exit_1']                      | 45.0       | 200
C      | ['Exit_2']                      | 12.0       | 10

--- Final Load per Edge (Congestion Status) ---
Exit_1              : 220 people  [!!! CONGESTED !!!]
Exit_2              : 100 people
Exit_3              : 100 people
Stair_1_2-1         :  20 people
Stair_1_3-2         :  20 people
Stair_2_2-1         :  60 people
Stair_2_3-2         :  20 people
Stair_3_2-1         : 100 people
Stair_3_3-2         :  30 people
```

■ Results with front/backend

localhost:5173

Evacuation route optimization app

Enter the number of people

A: 30
B: 200
C: 10
D: 40
E: 60
F: 10
G: 20
H: 30
I: 20

Optimize execution

localhost:5173

Route Results

A → Exit_3
B → Exit_1
C → Exit_2
D → Stair_2_2-1 → Exit_2
It is → Stair_3_2-1 → Exit_3
F → Stair_2_2-1 → Exit_2
G → Stair_2_3-2 → Stair_2_2-1 → Exit_2
H → Stair_2_3-2 → Stair_2_2-1 → Exit_2
I → Stair_3_3-2 → Stair_3_2-1 → Exit_3

Edge congestion

Exit_1: 200 people
Exit_2: 110 people
Stair_2_3-2: 50 people
Stair_3_2-1: 80 people
Stair_1_2-1: 0 people
Exit_3: 110 people
Stair_1_3-2: 0 people
Stair_2_2-1: 100 people
Stair_3_3-2: 20 people

■ Results with front/backend

Enter the number of people

A: 30
B: 50
C: 30
D: 40
E: 60
F: 30
G: 30
H: 20
I: 20

Optimize execution

Route Results

A → Exit_2
B → Exit_3
C → Exit_1
D → Stair_2_2-1 → Exit_2
It is → Stair_3_2-1 → Exit_3
F → Stair_2_2-1 → Exit_2
G → Stair_3_3-2 → Stair_3_2-1 → Exit_3
H → Stair_3_3-2 → Stair_3_2-1 → Exit_3
I → Stair_2_3-2 → Stair_2_2-1 → Exit_2

Edge congestion

Exit_2: 110 people
Stair_2_2-1: 70 people
Exit_1: 220 people
Exit_3: 90 people
Stair_1_3-2: 20 people
Stair_3_3-2: 30 people
Stair_3_2-1: 90 people
Stair_1_2-1: 20 people
Stair_2_3-2: 20 people

■ Result / Creative points

- **Flexibility:** facilitate the flexible process even in dynamic network
- **Utility:** Compare with classical algorithm, it is more useful in practice.
- **Promptness:** Compare with classical algorithm, it is faster

Appendix : comparison between SA and QA

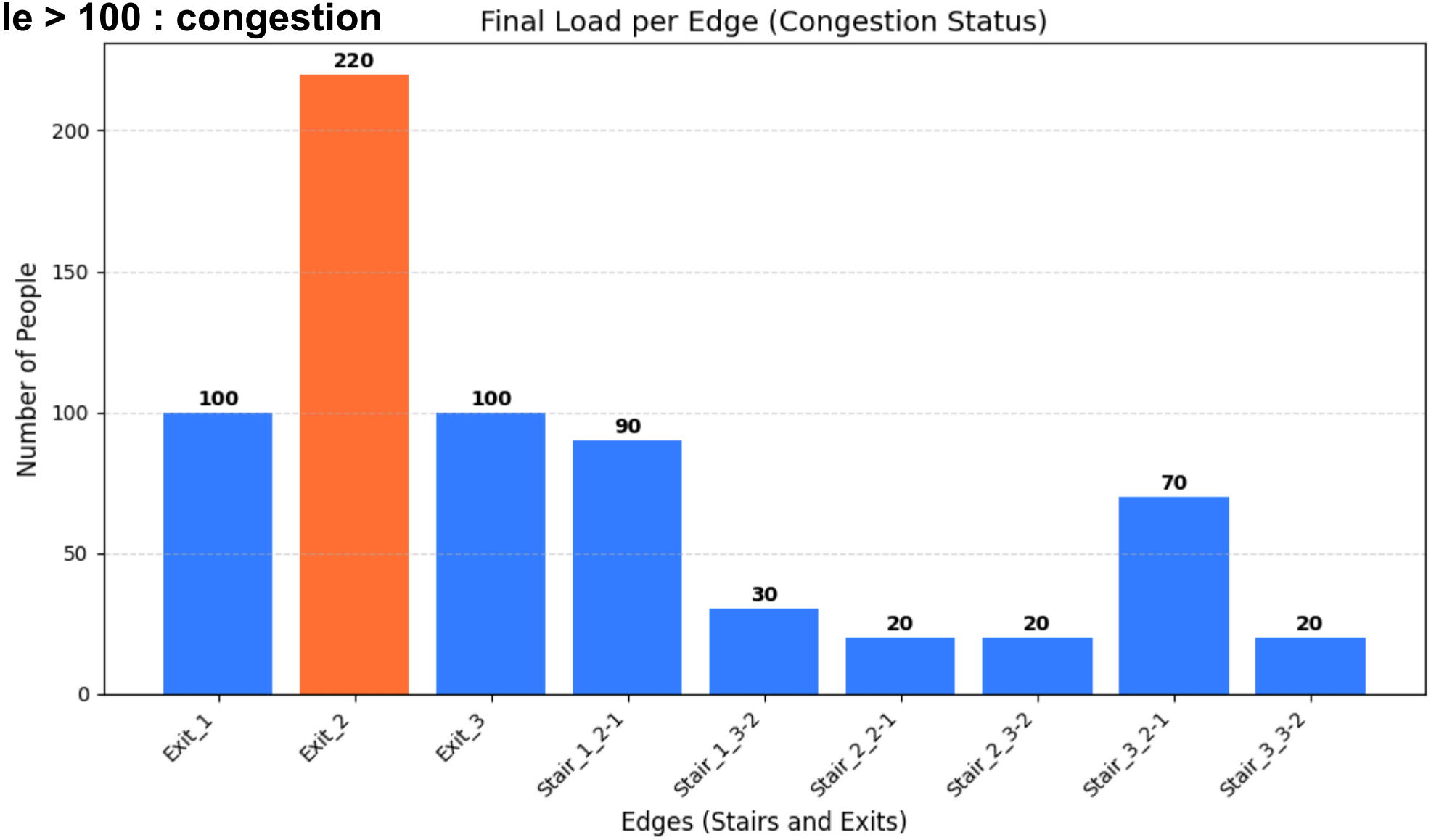
- **The list of initial number**
-

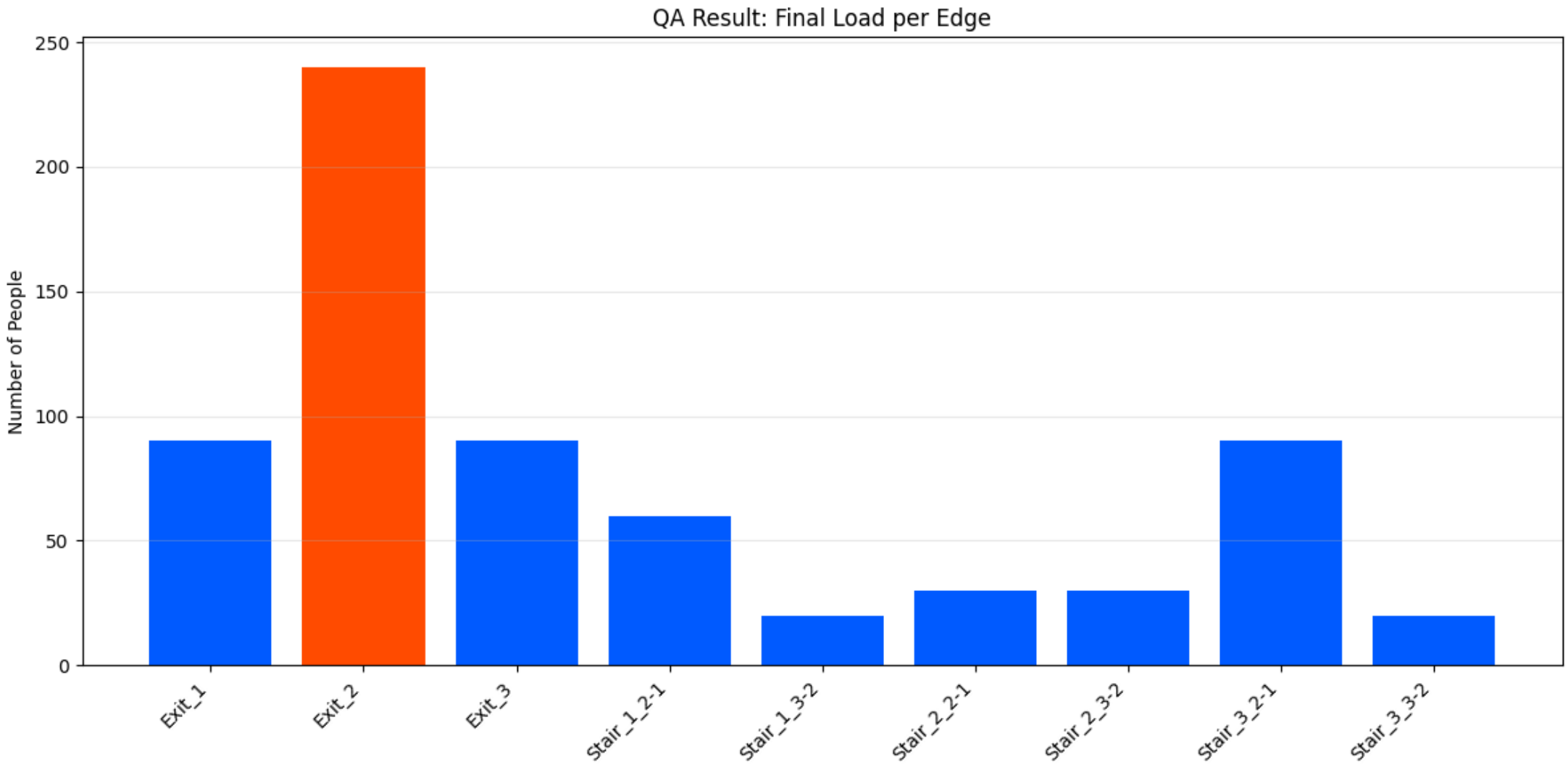
```
groups = {
    "G": 20, "H": 30, "I": 20,  # 3rd floor
    "D": 40, "E": 60, "F": 10,  # 2nd floor
    "A": 30, "B": 10, "C": 10  # 1st floor
}

# Base time for each route [Option 1, Option 2, Option 3]
base_times = {
    "3F": [20, 25, 30],  # Base times for 3rd floor groups
    "2F": [15, 20, 25],  # Base times for 2nd floor groups
    "1F": [5, 10, 15]    # Base times for 1st floor groups
}
```

Result of SA(Simulated Annealing)

If people > 100 : congestion



Result of QA(Quantum Annealing)

If people > 100 : congestion